DATAKERN ASSIGNMENT *www.data-kern.com*

Assignment 5: Olist Seller Scorecard (Spark Windows and Parquet)

Session: 6 (Spark, Level Up) · **Time:** 4 to 5 hours · **Difficulty:** Intermediate (Scaffolded)
Needs: Python 3.10+, Java 17+, PySpark 4.2 · **Cost:** free (runs on your laptop)

Scenario

Congratulations on your new job as a Data Engineer at **Olist**, a Brazilian marketplace. The Seller Success team has a problem: every Monday an analyst opens five huge CSV files, joins them by hand, and spends half a day producing a seller report. It is slow, and last month two analysts produced two different numbers for the same seller.

Your team lead hands you the task:

> "Build the foundation once. Read the raw CSVs, store them as a **proper Parquet table**, and produce three **trusted, repeatable** tables from it: the latest order per customer, the top 3 products per category, and a monthly scorecard per seller. If Spark skips the work it does not need, the Monday report will take seconds. And I need to be able to **prove** that it does."

You will use what you learned in Session 6: **window functions**, the **cost of UDFs**, **Parquet** and how it is laid out on disk, **partition pruning**, and **repartition vs coalesce**.

> **Why this matters:** a pipeline that "did not crash" is not a pipeline you can trust. In this assignment you will learn to check your Spark output against an independent answer and to read what Spark actually did from the physical plan.

> **AI use rule:** You may use AI tools (ChatGPT, Gemini, Claude, GitHub Copilot, Cursor, or similar) to help you write code. What matters is that you **understand and can reason about every line you submit**. AI can be confidently wrong, so treat its output as a suggestion from a fast but unreliable colleague, not as the answer: read it, run it, test it, and question it. If you cannot explain why your solution works, it is not yet your solution.

Use AI well (Do)	Do not
Ask it to *explain* a Spark error or a window frame, then re-explain it in your own words	Paste the assignment in and submit what comes back
Predict the output (row counts, number of files) before you run	Accept "it ran without errors" as proof
Verify against the self-check, the physical plan and the Spark UI	Trust the AI's confidence over your own evidence
Be ready to explain any line out loud	Submit code you could not defend in a review

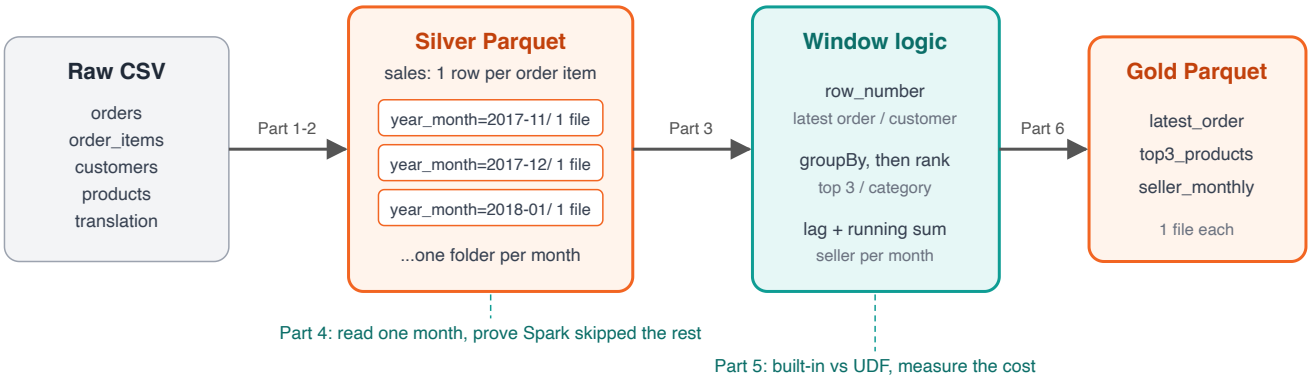
Objectives and deliverables

By the end you will be able to:

- read CSVs with **explicit schemas** and join them without losing or duplicating rows
- write a **tidy, partitioned Parquet** table and explain what is on disk
- solve "latest per group", "top N per group" and "running total" problems with **window functions**
- **prove** partition pruning from the physical plan
- measure the cost of a **Python UDF** against a built-in
- choose between `repartition`, `coalesce` and `partitionBy` for a given table

Deliverable	Where
Your finished pipeline	`starter/pipeline.py`
Self-check output (all green)	screenshot or paste in `REFLECTION.md`
Your answers with evidence from your runs	`starter/REFLECTION.md`
Git history with one commit per part	your repository

Mental model



Question	Memory partitions	Disk partitions
What are they?	Slices of the DataFrame processed by parallel tasks	Folders like `year_month=2018-03/`
Controlled by	`repartition(n)` / `repartition(col)` / `coalesce(n)`	`.write.partitionBy(col)`
Cost to change	`repartition` shuffles, `coalesce` does not (only reduces)	Decided when you write; hard to change later
What goes wrong	Too few = idle cores; one huge = a straggler	Too many tiny files = slow reads

Tool	Use it when	Remember
`groupBy().agg()`	you want one row per group	collapses the rows
Window function	you want to keep every row **and** compare it with its group	needs `partitionBy`, or one task gets everything
Built-in function	always, first	Spark can optimise it
Python UDF	no built-in exists	opaque to Spark, runs Python per row

Setup

You can reuse the Olist CSV files from the earlier assignments.

1. Folder and virtual environment (from the `starter/` folder):

```
cd starter
python -m venv .venv
source .venv/bin/activate          # Windows PowerShell:  .venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

2. Data. Create `data/olist/` inside `starter/` and copy these 5 files into it (or set `OLIST_DIR` to the folder that holds them):

`olist_orders_dataset.csv` · `olist_order_items_dataset.csv` · `olist_customers_dataset.csv` · `olist_products_dataset.csv` · `product_category_name_translation.csv`

3. VS Code. Press `Ctrl+Shift+P` (Mac: `Cmd+Shift+P`) → **Python: Select Interpreter** → choose the `.venv` one. Open a new terminal: the prompt must start with `(.venv)`.

4. Java. Spark needs Java 17 or newer:

```
java -version
```

5. Windows only: Spark needs `HADOOP_HOME` with `bin\winutils.exe`, the same as in the Session 6 demos.

Check	Expected
`python --version`	3.10 or newer, from `.venv`
`python -c "import pyspark; print(pyspark.__version__)"`	`4.2.0`
`java -version`	17 or newer
`python check_my_work.py`	prints `TODO` lines (nothing is written yet), no crash

> If Spark fails with `Can't assign requested address`, set `SPARK_LOCAL_IP=127.0.0.1` (Windows PowerShell: `$env:SPARK_LOCAL_IP="127.0.0.1"`) and retry.

```
git init            # if this folder is not a repository yet
git add -A && git commit -m "Assignment 5: starter kit"
```

What is given and what you write

File	Who	What
`helpers.py`	given	`get_spark`, `csv_path`, `fresh_dir`, `uri`, `count_files`, `folder_size_mb`, `plan_text`, `timed`
`pipeline.py`	**you**	10 functions marked `TODO`; the `main()` driver is given
`check_my_work.py`	given	compares your output with an independent pandas answer
`REFLECTION.md`	**you**	questions that need evidence from your runs

The skeleton you receive in `pipeline.py` (each body is `raise NotImplementedError`; the docstrings hold the full specification):

```
def build_sales(spark): ...                        # Part 1  -> DataFrame (SALES_COLUMNS)
def write_silver(sales, silver_path): ...          # Part 2a -> Parquet, year_month=...
def compare_csv_parquet(sales, workdir): ...       # Part 2b -> dict of sizes and files
def latest_order_per_customer(sales): ...          # Part 3a
def top3_products_per_category(sales): ...         # Part 3b
def seller_monthly(sales): ...                     # Part 3c
def read_month(spark, silver_path, year_month): ...  # Part 4
def add_price_band_builtin(df): ...                # Part 5a
def add_price_band_udf(df): ...                    # Part 5b
def write_gold(tables, gold_path): ...             # Part 6
```

Open `helpers.py` and read it before you start. Every function in `pipeline.py` has a docstring that is the **specification**: columns, tie-break rules, what must be true.

Explore first (before you write any pipeline code)

Create a notebook `explore.ipynb` in `starter/` (select the `.venv` kernel) and run these cells one by one. Use the Spark UI address printed by `get_spark()` (usually http://localhost:4040).

```
from helpers import *
spark = get_spark("explore")
```

```
# 1. what does a CSV look like to Spark WITHOUT a schema?
raw = spark.read.csv(csv_path("olist_order_items_dataset.csv"), header=True, inferSchema=True)
raw.printSchema()
```

```
# 2. look at the data you will join
for f in ["olist_orders_dataset.csv", "olist_customers_dataset.csv",
"olist_products_dataset.csv"]:
    spark.read.csv(csv_path(f), header=True).show(3, truncate=30)
```

```
# 3. products without an English category? (these will matter for your join)
prod = spark.read.csv(csv_path("olist_products_dataset.csv"), header=True)
tr = spark.read.csv(csv_path("product_category_name_translation.csv"), header=True)
print("products:", prod.count(), "| no category:", prod.filter("product_category_name is
null").count())
prod.join(tr, "product_category_name", "left").filter("product_category_name_english is
null").count()
```

PAUSE AND OBSERVE #1. Open the Spark UI **Jobs** tab. Cell 1 created a job even though you never asked for rows (it is `inferSchema` reading the whole file). Write down: how many jobs ran for cell 1, and why would a pipeline that runs every Monday prefer an explicit schema? Then, in cell 3: how many products have no category, and how many have a category that is not in the translation table? What would happen to those rows if you used an **inner** join?

Part 1: Build the `sales` table

Implement `build_sales(spark)` in `pipeline.py`. Read the five files with the given DDL schemas (`header=True`, `schema=...`) and join them into one wide table with exactly the columns in `SALES_COLUMNS`: **one row per order item**.

Hints you are allowed to have:

- items → orders on `order_id`; → customers on `customer_id`; → products on `product_id`; → translation on `product_category_name`
- some joins must keep rows that have no match, others must not drop anything: decide which join type each one needs, and say why
- `F.coalesce(col, F.lit("unknown"))` gives you a default

```
python check_my_work.py       # test 1 must pass before you continue
git add -A && git commit -m "Part 1: build_sales with explicit schemas"
```

PAUSE AND OBSERVE #2. Your `sales` table must have exactly as many rows as the items file. Before you run the check, predict which of your joins could lose rows. Then **change the `products` join and the `translation` join to inner joins, one at a time**, and report the row count each time. Which one loses rows on this data, which one does not, and why? Would anyone notice the missing rows in a dashboard?

Part 2: Write Parquet the tidy way

2a. `write_silver`

Write `sales` as Parquet into `output/silver/sales/`, with one folder per month and **exactly one file per folder**:

```
silver/sales/year_month=2017-11/part-....parquet
silver/sales/year_month=2017-12/part-....parquet
```

Add a `year_month` column (text such as `2018-03`) from `purchase_ts`. The tidy-write pattern from Session 6 has two ingredients, one for memory partitions and one for disk partitions. Which two calls are they, and in what order?

2b. `compare_csv_parquet`

Write `sales` as CSV and as plain Parquet, and return the sizes and file counts measured on disk.

```
python check_my_work.py
git add -A && git commit -m "Part 2: partitioned Parquet silver table and CSV vs Parquet sizes"
```

Look inside a Parquet file (run in your notebook)

```
import pyarrow.parquet as pq, glob
f = glob.glob("output/silver/sales/year_month=2018-03/*.parquet")[0]
meta = pq.ParquetFile(f).metadata
print("row groups:", meta.num_row_groups, "| rows:", meta.num_rows)
for i in range(meta.num_columns):
    col = meta.row_group(0).column(i)
    if col.path_in_schema == "price":
        print(col.path_in_schema, col.statistics.min, col.statistics.max, col.compression)
```

PAUSE AND OBSERVE #3. Report the sizes on **your** machine: CSV, plain Parquet, and the partitioned silver table. Is the partitioned table larger or smaller than the plain one, and why? Where in the file did Spark find `min` and `max` for `price`, and how could a query like `price > 500` use them to **skip** a file without reading it?

Part 3: Window functions

Implement three functions. All three receive the silver data (the driver reads it back from Parquet).

Function	Idea	Specification highlights
`latest_order_per_customer`	`partitionBy` + `orderBy` + `row_number`	key is `customer_unique_id` (not `customer_id`: check why in the data); ties broken by `order_id` descending
`top3_products_per_category`	`groupBy` **first**, window **second**	revenue = `round(sum(price), 2)`; ties broken by `product_id` A to Z
`seller_monthly`	`lag` + running total with an explicit frame	`prev_month_revenue`, `mom_growth_pct`, `running_revenue` (see the docstring for the exact rounding)

```
python check_my_work.py
git add -A && git commit -m "Part 3: window functions (latest order, top 3, seller monthly)"
```

PAUSE AND OBSERVE #4. (a) Run `sales.select("customer_id").distinct().count()` and the same for `customer_unique_id`. Which one would give a wrong "latest order per customer", and what did *your* numbers show? (b) For `top3`, order by `revenue` only (no `product_id` tie-break) and use `dense_rank`: how many rows do you get compared with `row_number`, and what does that tell you about ties in this data? (c) In `seller_monthly`, the first month of every seller has `null` in two columns. Why is that correct, and what would a `0` have done to the growth number?

Part 4: Prove partition pruning

Implement `read_month(spark, silver_path, year_month)` and return a DataFrame that reads **only the folder for that month**. Then look at the proof:

```
df = read_month(spark, "output/silver/sales", "2018-03")
print(plan_text(df))      # find the line that starts with FileScan parquet
```

Find `PartitionFilters: [...]`, and the number of files and size Spark reads. Then open the Spark UI **SQL** tab, run `df.count()`, click your query and read the scan node: `number of files read`.

```
python check_my_work.py
git add -A && git commit -m "Part 4: read_month with partition pruning"
```

PAUSE AND OBSERVE #5. Paste the `PartitionFilters` line. Now write a second version that filters on `date_format(purchase_ts, 'yyyy-MM')` instead of the `year_month` column and compare the two plans: which one reads all the folders and why? What does this tell you about *how* you must write a filter for pruning to work?

Part 5: Built-in vs UDF

Implement `add_price_band_builtin` and `add_price_band_udf`. Both add a `price_band` column: `budget` (price < 20), `mid` (20 to under 100), `premium` (100 and above). One must use built-in functions only; the other must be a Python UDF.

Then run `python pipeline.py` and read line `[5]` for the two timings. Run it three times: the first run includes warm-up.

```
python check_my_work.py
git add -A && git commit -m "Part 5: price band as built-in and as UDF"
```

PAUSE AND OBSERVE #6. Compare `plan_text` of the two versions. What node appears only in the UDF plan, and what does it tell you about where the work happens? Record your timings (all three runs) and the ratio. The old textbook rule says a Python UDF is 10x slower or worse. Is that what you measured on this Spark version, and does your result change your advice about when to write a UDF?

Part 6: Write the gold tables

Implement `write_gold(tables, gold_path)`. The three gold tables are small and are read whole by analysts: each must be written as Parquet with **exactly one file** and **no partition folders**.

```
python pipeline.py           # the whole pipeline, start to finish
python check_my_work.py
git add -A && git commit -m "Part 6: gold tables as single tidy Parquet files"
```

PAUSE AND OBSERVE #7. Why is one file the right answer for `seller_monthly` (around 16 thousand rows) but would be the wrong answer for `silver/sales` at 10 GB? You can get one file with `coalesce(1)` or `repartition(1)` : they are **not** the same. What does each do to the stage that runs *before* the write, and which one would you pick here?

Validate

A script that did not crash is **not** validated. Do both of these.

1. Self-check

```
python check_my_work.py
```

It must end with `9 passed, 0 failed, 0 still TODO`. It compares your output with an independent pandas computation. Read the failures: each one says what is wrong, not how to fix it.

2. Ground truth: compare with the source

Your numbers on the real Olist files must be close to these. If one is off, **find out why** before you touch the code.

What	Expected
Rows in `sales`	112,650 (= rows in the items file)
Parquet files in `silver/sales`	24, one per `year_month=` folder
Rows in `latest_order`	95,420 (one per distinct `customer_unique_id`)
Rows in `top3_products`	213
Rows in `seller_monthly`	16,441
Rows for `2018-03`	8,217

Now pick **one** seller and one customer and verify them by hand with Spark SQL straight on the CSVs (not on your own tables):

```
spark.read.csv(csv_path("olist_order_items_dataset.csv"), header=True, inferSchema=True) \
    .createOrReplaceTempView("items")
spark.sql("SELECT seller_id, COUNT(*), ROUND(SUM(price),2) FROM items GROUP BY seller_id ORDER BY
3 DESC LIMIT 3").show()
```

Compare the largest seller's `running_revenue` in the **last** month with this total. They must match. If they do not, what did your window frame do?

Break it on purpose

Change **one** thing, **predict**, observe, **explain**, then revert (`git checkout .`). Write your findings in `REFLECTION.md` .

#	Change	Predict first	Look at
1	Remove `partitionBy` from the `latest_order_per_customer` window (keep the `orderBy`)	How many rows come back? What will Spark do with the data?	the WARN line in the terminal; Spark UI → Stages: number of tasks in the window stage
2	In `write_silver` , remove the `repartition(...)` call	How many files per folder?	`count_files` per folder in `output/`
3	In a **scratch copy** of the data, `partitionBy("seller_id")`	How many folders? How big is each file?	`output/scratch/` (folders, average file size)
4	Read silver, filter, then `.write.mode("overwrite")` to the **same path** you read from (do this on a scratch copy only)	Will it work? Can you still read the folder afterwards?	the first error that names a file; then the folder: is anything left?
5	Make `read_month` filter on `purchase_ts` instead of `year_month`	Same rows? Same files read?	`plan_text` and the SQL tab

> **Never run experiment 4 on your only copy of the data.** Copy `output/silver/sales` to `output/scratch/` first. Experiment 4 is the reason Delta Lake exists: be ready to explain it.

Reflection

Fill in `starter/REFLECTION.md` . Every answer needs a **number or a line from your own runs**, not a textbook sentence.

Stretch goals (optional)

- Make `seller_monthly` **calendar-aware**: a seller who skipped a month should show growth against the previous *calendar* month (null when the previous month had no sales). What is the cost of the extra step?
- Add a `delivery_days` column (`order_delivered_customer_date` minus `order_purchase_timestamp`) and a window that ranks sellers by average delivery time **within each state**.
- Rewrite the price band as a **pandas UDF** and measure it as a third line in your table.
- Write `silver` partitioned by `year` and `month` as two columns, and explain how a filter on `year_month` changes.

Submission checklist and rubric

☐ `python check_my_work.py` ends with `9 passed, 0 failed, 0 still TODO`

☐ `python pipeline.py` runs start to finish on a clean `output/` folder

☐ One commit per part, with meaningful messages

☐ All seven **Pause and observe** answers and the five experiments are in `REFLECTION.md` with evidence from your runs

☐ You can explain every line you submit out loud without looking at it

Criterion	Points	What earns them
Correctness	30	the self-check is green; ground-truth numbers match
Design	20	the right join types, window specs, partition choices; no needless shuffles or UDFs
Safety and reasoning	15	tie-breaks, null handling, no data loss in joins; experiment 4 explained
Validation habit	20	hand-verified a seller and a customer; used `plan_text` and the Spark UI as evidence
Reflection quality	15	specific, with numbers from your own runs

Don’t focus on getting the answer. Focus on learning how to engineer the solution.

— Uma Kiran | DataKern | www.data-kern.com